Martin Kerstenne Groupe 4.B

Rapport SAE

S1.02 - Démonstration d'algorithmes

Introduction

Objectif du projet : comparer les performances de 6 tris différents selon la taille et le type de données.

Type de données : afin d'évaluer plus précisément le temps d'exécution des algorithmes en fonction des types de données, trois sortes de vecteurs différents, de tailles n variant de 1000 à 100 000 éléments (avec un pas de 1000 éléments), ont été utilisés.

Les valeurs de ces vecteurs sont des entiers compris entre 0 et 1024n :

Type 1 (aléatoire) : tous les éléments sont générés aléatoirement, de manière uniforme.

Type 2 (partiellement trié) : la première moitié du vecteur est quasi-triée suivant la formule v[i] = 1024 * i + r (avec r une valeur aléatoire comprise entre 0 et 1023), tandis que la seconde moitié reste aléatoire.

Type 3 (Partiellement inversé) : la première moitié suit une progression décroissante selon v[i] = 1024 * (n - i) + r, la seconde moitié étant aléatoire.

Rappel : L'ensemble des tris et du projet en général ont été faits sous Linux (Ubuntu) en utilisant le compilateur g++ avec l'option –Ofast.

Le temps d'exécution est mesuré en microsecondes à l'aide de la bibliothèque <chrono>.

Algorithmes de tri choisis :

- std::sort
- std::stable_sort
- Tri par sélection
- Tri par insertion
- Tri rapide (ou quicksort)
- Une version modifiée du tri rapide (ici modifquicksort)

Tous ces tris ont été séparé en 2 catégories distinctes en fonction de leur complexité respective. La première catégorie regroupe les tris ayant une complexité de l'ordre de n^2 ou quadratique (plus lents). La deuxième quant-à-elle, regroupe les tris ayant une complexité de l'ordre de n*log(n) ou logarithmique (plus rapides).

Tris en $O(n^2)$:

- **Tri par sélection :** le principe est de parcourir le vecteur pour trouver le plus petit élément et l'échanger avec celui à la position actuelle, puis de répéter l'opération pour le reste du tableau. Sa complexité ne change en aucun cas, ce qui le rend inefficace pour les grands vecteurs.

- **Tri par insertion :** cet algorithme parcourt le vecteur et place chaque nouvel élément à sa position correcte parmi les éléments déjà triés précédemment. Bien que sa complexité moyenne soit en $O(n^2)$, il est assez performant sur des données presque triées (comme les vecteurs de type 2), où il peut atteindre une complexité proche du linéaire (O(n)).

Tris en O(n*logn) :

- **std::sort :** obtenable via la bibliothèque <algorithm> de C++. Il utilise généralement une implémentation que l'on peut qualifier

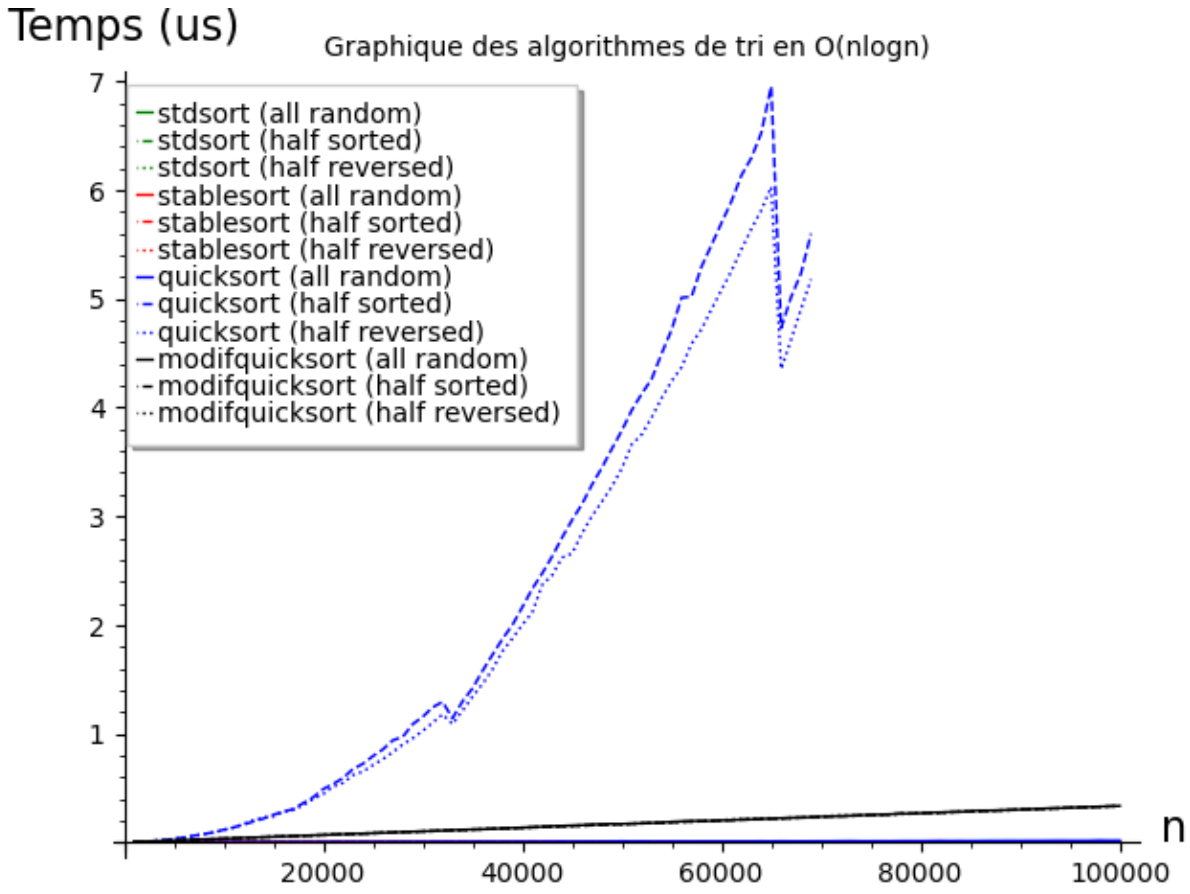
“d’hybride” qui combine plusieurs tris pour garantir une performance optimale.

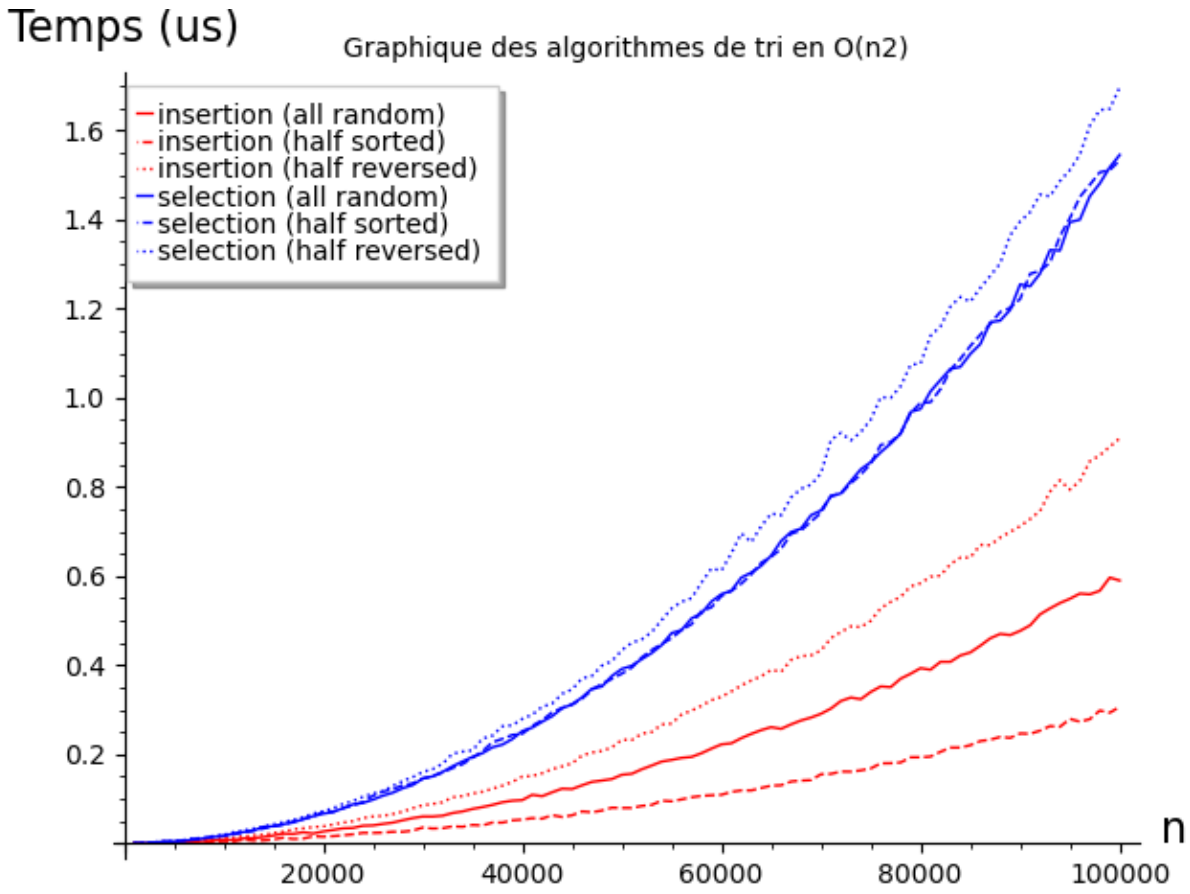
- **std::stable_sort :** obtenable via la bibliothèque <algorithm> de C++, ce tri est similaire à std::sort. Cet algorithme a la particularité d'être stable, ce qui signifie qu'il préserve l'ordre d'origine des éléments ayant des valeurs identiques. Il est particulièrement utile lorsque l'ordre initial des données possède une importance.

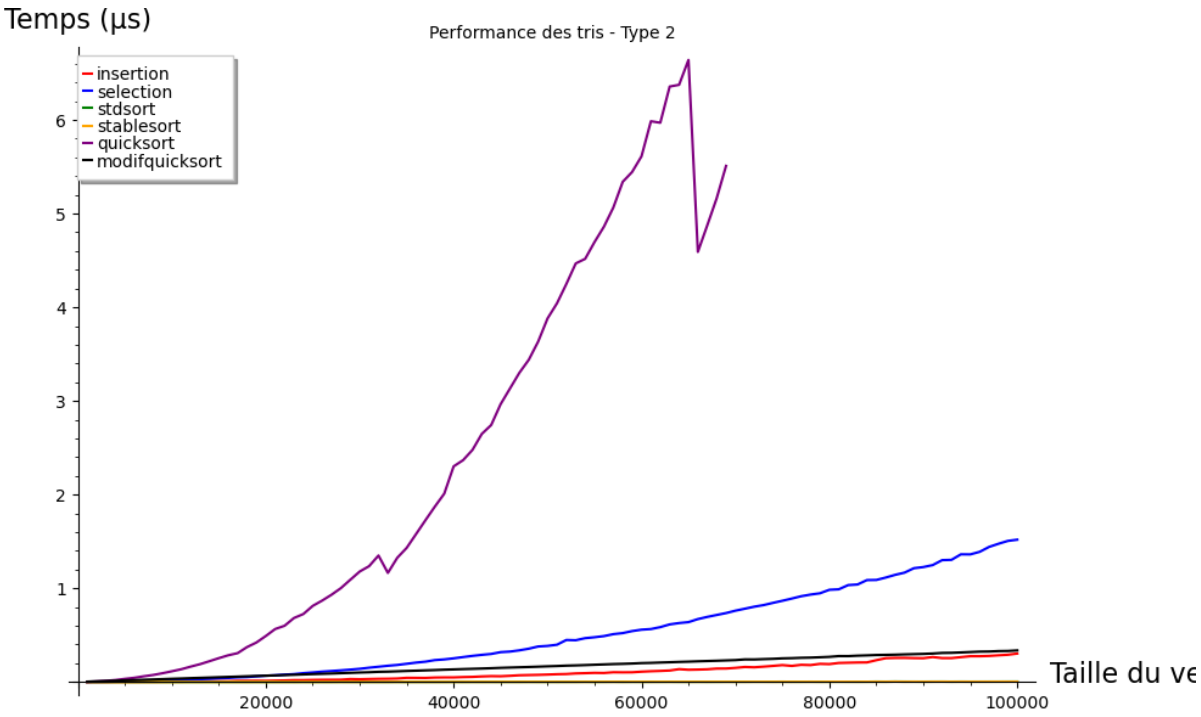
- **Quicksort : c**et algorithme utilise le paradigme de programmation "diviser pour régner". Il choisit systématiquement le premier élément du vecteur comme pivot. On répartit les autres éléments du vecteur dans trois groupes : ceux plus petits que le pivot, ceux égaux au pivot, et ceux plus grands que le pivot. On applique ensuite récursivement la même méthode aux groupes d'éléments plus petits et plus grands. Bien que très rapide sur des données aléatoires, cette stratégie est risquée : sur des vecteurs déjà triés ou inversés, la complexité passe en $O(n^2)$ et peut provoquer des erreurs.

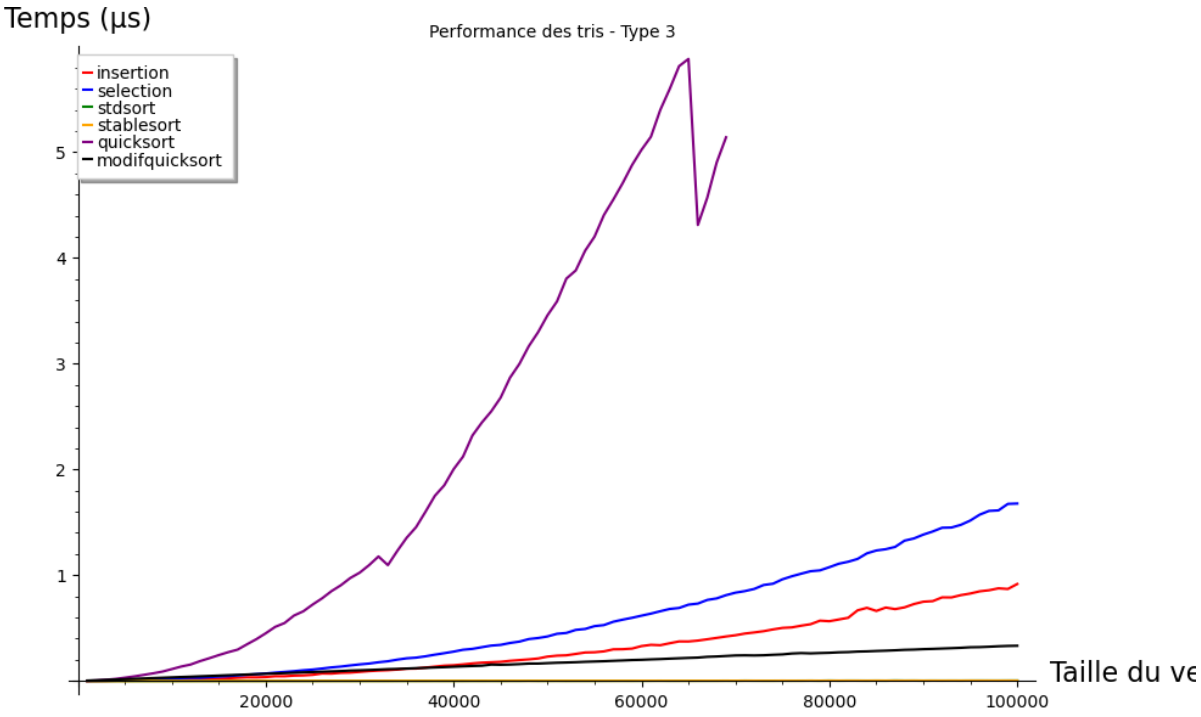
- **Quicksort modifié :** C'est une variante de l'algorithme précédent où le pivot est choisi de manière aléatoire. Cette “petite” modification permet d'éviter les pires scénarios de performance et assure un temps d'exécution stable quel que soit le type de vecteur utilisé.

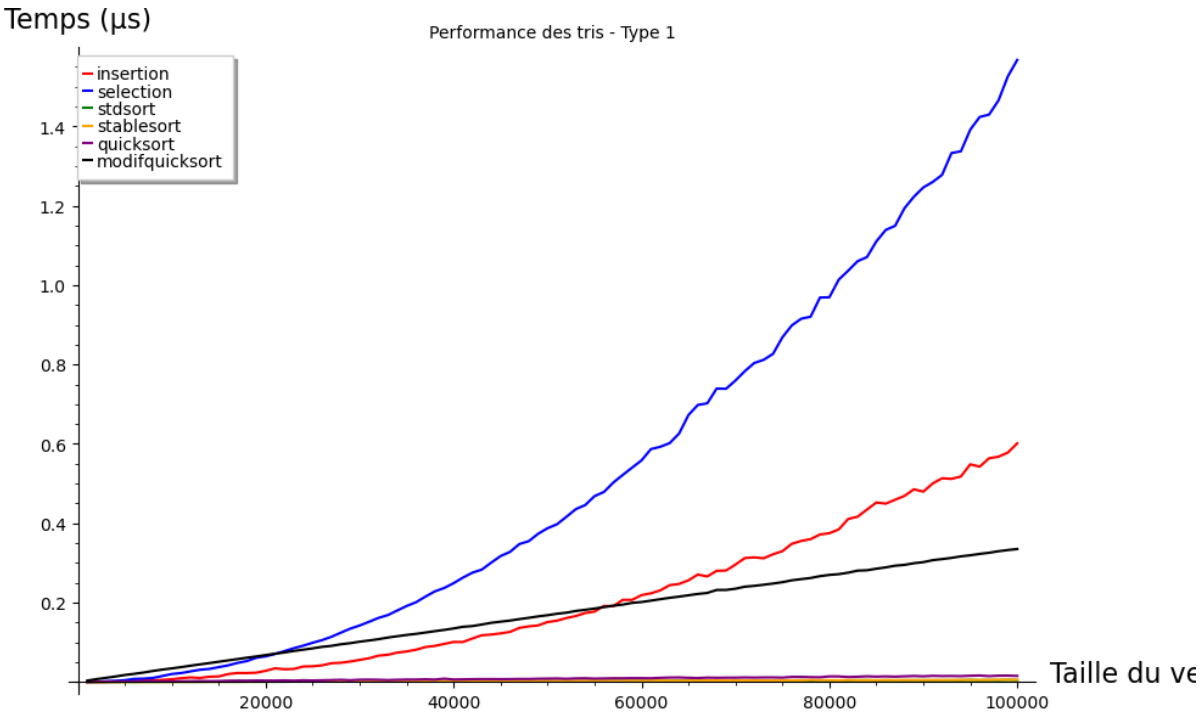
Problème d’exécution : lors des tests pour le tri quicksort, les vecteurs de type 2 et 3 étant trop lourds pour l’algorithme, un “timeout” s’est appliqué afin de ne pas bloquer le reste de l’exécution. Il n’y a donc un manque donné pour le tri quicksort lorsque la taille est aux alentours de 70 000 éléments.


Temps (us)
Graphique des algorithmes de tri en O(nlogn)
stdsort (all random)
stdsort (half sorted)
stdsort (half reversed)
stablesort (all random)
stablesort (half sorted)
stablesort (half reversed)
quicksort (all random)
quicksort (half sorted)
quicksort (half reversed)
modifquicksort (all random)
modifquicksort (half sorted)
modifquicksort (half reversed)
7
6
5
4
3
2
1
20000
40000
60000
80000
100000
n


Temps (us)
Graphique des algorithmes de tri en O(n2)
insertion (all random)
insertion (half sorted)
insertion (half reversed)
selection (all random)
selection (half sorted)
selection (half reversed)
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
20000
40000
60000
80000
100000
n


Temps (µs)
Performance des tris - Type 2
insertion
selection
stdsort
stablesort
quicksort
modifquicksort
6
5
4
3
2
1
20000
40000
60000
80000
100000
Taille du ve


Temps (µs)
Performance des tris - Type 3
insertion
selection
stdsort
stablesort
quicksort
modifquicksort
5
4
3
2
1
20000
40000
60000
80000
100000
Taille du ve


Temps (µs)
Performance des tris - Type 1
insertion
selection
stdsort
stablesort
quicksort
modifquicksort
1.4
1.2
1.0
0.8
0.6
0.4
0.2
20000
40000
60000
80000
100000
Taille du ve